

SDK 2.x版本到3.x版本升级指引

- 腾讯云语音识别SDK v3.0版本进行了一次重构优化，如果您的应用曾经集成了2.x版本的SDK，可以阅读本文档，本文档将列出升级所需要关注的点；

1、SDK模块变动

为减小集成SDK的体积增量，SDK已经对 实时语音识别、一句话识别、录音文件识别、语音合成 模块进行了解耦，SDK目录中的aar有如下对应关系：

- asr-file-recognize-release.aar :录音文件识别极速版(SDK内仍然保留普通版录音文件识别接口，建议升级录音文件识别极速版，未来SDK将会移除普通版录音文件识别接口)
- asr-one-sentence-release.aar:一句话识别
- asr-realtime-release.aar : 实时语音识别

注意：语音合成TTS SDK已经分离，如果您还使用了2.x版本内置的语音合成TTS SDK, 请访问 <https://console.cloud.tencent.com/tts/download> 下载

2、接口变动

- 录音文件识别极速版

接口无变化，依赖发生变化，不再依赖okhttp,导入如下依赖

```
implementation 'com.google.code.gson:gson:2.8.5'
```

- 一句话识别

1、接口无变化，依赖发生变化，不再依赖okhttp,导入如下依赖

```
implementation 'com.google.code.gson:gson:2.8.5'
```

2、音频格式类型 QCloudOneSentenceRecognitionParams.setVoiceFormat参数类型由枚举改为String，SDK内部不再做限制，您可以直接传字符串来指定音频文件类型，支持的类型参数详情请看API文档 <https://cloud.tencent.com/document/product/1093/35646> VoiceFormat参数；

- 实时语音识别

- 1、依赖

```
implementation 'com.squareup.okhttp3:okhttp:4.2.2'
```

- 2、识别结果回调监听变动:

```
public interface AudioRecognizeResultListener {

    /**
     * 返回分片识别结果（非稳态结果，识别过程中会持续修正）：
     *
     * 实时返回方式下会回调，尾包返回方式下不会回调
     * @param request
     * @param result
     * @param order 该分片对应的语音流在所有语音流中的次序（0，1，2...）
     */
    void onSliceSuccess(AudioRecognizeRequest request, AudioRecognizeResult result,
        int order);

    /**
     * 返回语音流识别结果（稳态结果，每句话的最终识别结果，一次识别有多少句话由后端算法切分确定，
     如有多句话会返回多次）：
     *
     * @param request
     * @param result
     * @param order 该语音流在所有语音流中的次序（0，1，2...）
     */
    void onSegmentSuccess(AudioRecognizeRequest request, AudioRecognizeResult
        result, int order);

    /**
     * 识别结束回调，返回所有的识别结果
     * @param request 相应的请求
     * @param result 识别结果，sdk内会把所有的onSegmentSuccess结果合并返回，如果业务不需要，可
     以只使用onSegmentSuccess返回的结果
     *
     * 注意：仅收到onStopRecord回调时，表明本次语音流录音任务已经停止，但识别任务还未停止，需
     要等待后端返回最终识别结果，
     *
     * 如果此时立即启动下一次录音，结果上一次结果仍会返回，可以调用
     cancelAudioRecognize取消上一次识别任务
     *
     * 当收到 onSuccess 或者 onFailure时，表明本次语音流识别完毕，可以进行下一次识
     别；
     */
    void onSuccess(AudioRecognizeRequest request, String result);
```

```

/**
 * 识别失败
 * @param request 相应的请求
 * @param clientException 客户端异常
 * @param serverException 服务端异常
 * @param response 服务端返回的json字符串（如果有）
 * 注意：仅收到onStopRecord回调时，表明本次语音流录音任务已经停止，但识别任务还未停止，需要等待后端返回最终识别结果，
 * 如果此时立即启动下一次录音，结果上一次结果仍会返回，可以调用
cancelAudioRecognize取消上一次识别任务
 * 当收到 onSuccess 或者 onFailure时，表明本次语音流识别完毕，可以进行下一次识别；
 */
void onFailure(AudioRecognizeRequest request, ClientException clientException,
ServerException serverException, String response);
}

```

3、识别结果类AudioRecognizeResult 变化：新增String getResultJson() 接口，可通过此方法获取后端返回的未做解析的json原文本，如有需求，可拿到后按业务需求自定义处理。

4、支持设置自学习模型 id ，接口：AudioRecognizeRequest.setCustomizationId("")

5、取消识别任务接口：移除了任务id概念，取消任务可直接调用AAIClient.cancelAudioRecognize()，无需传入任务id;

6、错误码变更，详见"实时语音识别Android SDK.md" 文档；

7、识别状态监听接口AudioRecognizeStateListener新增 静音检测回调

```

/**
 * 静音检测回调
 * 当设置AudioRecognizeConfiguration setSilentDetectTimeOut为true时，如触发静音超时，将触发此回调
 * 触发此回调相当于手动调用了 aaiClient.stopAudioRecognize()
 */
@Override
public void onSilentDetectTimeOut() {
//您的业务逻辑
}

```

8、自定义音频源接口变动(移除了maxLengthOnceRead接口实现，您无需再实现此接口);

```

/**
 * 音频数据源：用户初始化实例后，其中的方法用户不可主动调用
 */
public interface PcmAudioDataSource {

```

```

/**
 * 将长度为length的语音数据从下标0依次放入到audioPcmData数组中。
 *
 * @param audioPcmData 音频数据缓存数组
 * @param length 理论缓存数据的长度（最大长度），这里的长度对应的时长与
AudioRecognizeConfiguration.sliceTime(xxx) * 设置的分片时长一致
 * @return 实际缓存数据的长度 -1表示结束
 * 注意：如果您使用自定义数据源（例如第三方推流获取的音频流），而非录音器，如果为了最好的识别
效果，应尽量保证每次读取的数据 * 长度都填满length
 * 如果您没有调整AudioRecognizeConfiguration.sliceTime()设置的分片时长，SDK默认40ms，
如果您要调整length的长度，应 * 调用AudioRecognizeConfiguration.sliceTime()调整分片时
间
 * 16000 采样率40ms，长度计算方式--->对应数据长度 = 16000 * (40/1000) * (16/8*1) =
1280 (length=采样率 * 采样 * 时间 * 采样位数/8*通道数 (Bytes))
 */
int read(short[] audioPcmData, int length);

/**
 * 用户在开始识别后，sdk会主动调用该方法，用户可以在这里做一些初始化的工作
 * @throws ClientException
 */
void start() throws ClientException;

/**
 * 用户在停止识别后，sdk会主动调用该方法，用户可以在这里做一些数据清理工作
 */
void stop();

/**
 * 是否保存语音源文件的开关，打开后，音频数据将通过onNextAudioData回调返回给调用层；
 */
boolean isSetSaveAudioRecordFiles();
}

```